



DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA
ED ELETTRONICA E SISTEMISTICA

Corso di Laurea Magistrale in Ingegneria Informatica

Relazione Progetto Intelligenza Artificiale

Autori

Francesco Carmelo Capria (263664)
Giuseppe Mattia Greco (264526)
Emanuele Vita (264140)

Docenti

Prof. Francesco Scarcello
Prof. Davide Mario Longo

A. A. 2024/2025

Indice

1	Introduzione	1
1.1	Contesto e Motivazione	1
1.2	Obiettivi del Progetto	1
1.3	Contributi	2
2	Analisi dei requisiti	3
2.1	Introduzione	3
2.2	Requisiti funzionali	3
2.2.1	RF1: Generazione della lore	3
2.2.2	RF2: Generazione file PDDL	4
2.2.3	RF3: Validazione PDDL	4
2.2.4	RF4: Reflective Agent	4
2.2.5	RF5: Validazione del piano da parte dell'utente	5
2.2.6	RF6: Generazione della storia narrativa	5
2.2.7	RF7: Interfaccia utente Web (Flask)	5
2.3	Requisiti non funzionali	5
2.3.1	Sicurezza (RNF1)	6
2.3.2	Affidabilità (RNF2)	6
2.3.3	Performance (RNF3)	6
2.3.4	Usabilità (RNF4)	6
2.3.5	Manutenibilità (RNF5)	6
2.3.6	Portabilità (RNF6)	6
2.3.7	Scalabilità (RNF7)	6
2.4	Casi d'uso	7
2.4.1	UC1: Generazione quest completa	7
2.4.2	UC2: Correzione manuale PDDL	7
2.4.3	UC3: Utilizzo RAG per contesto esterno	7
2.5	Vincoli e dipendenze	7
3	Tecnologie utilizzate	8

3.1	Panoramica	8
3.2	Core language e orchestrazione	8
3.2.1	Python	8
3.3	Pipeline LLM e orchestrazione AI	8
3.3.1	LangChain	8
3.3.2	LLM: Google Gemini 2.5 Flash	8
3.4	RAG e gestione della conoscenza	9
3.4.1	Caricamento e segmentazione documenti (PyPDF)	9
3.4.2	Vector store e retrieval (FAISS + embeddings)	9
3.5	Pianificazione formale e validazione	9
3.5.1	Standard PDDL	9
3.5.2	Planner: Fast-Downward	9
3.5.3	Validazione: VAL	10
3.6	Infrastruttura web e configurazione	10
3.6.1	Backend web: Flask	10
3.6.2	Persistenza e gestione configurazione	10
4	Sviluppo del progetto	11
4.1	Componente RAG e generazione della lore	11
4.1.1	Strategie di Prompting e Sicurezza	11
4.1.2	Struttura della lore come ponte verso il PDDL	12
4.2	Generazione PDDL e validazione	12
4.2.1	Esecuzione del planner e validazione del piano	13
4.2.2	Esempio minimale di dominio	13
4.2.3	Classi di errori e strategia di correzione	14
4.3	Esecuzione tramite Flask	14
4.3.1	Interazione Frontend-Backend	15
4.4	Flusso operativo tipico	15
4.4.1	Interazioni dei componenti	17
4.4.2	Artefatti prodotti lungo la pipeline	18
4.5	Mapping requisiti funzionali — codice	18
5	Conclusioni	20
5.1	Risultati Ottenuti	20
5.2	Sfide e Limitazioni	20
5.2.1	Qualità e Coerenza della Generazione	21

5.2.2	Performance e Scalabilità	21
5.2.3	Dipendenze Esterne	21
5.3	Lavori Futuri	21
5.3.1	Miglioramenti alla Generazione	21
5.3.2	Valutazione e Validazione	22
5.3.3	Architettura e Deployment	22
5.4	Considerazioni Finali	22

Capitolo 1

Introduzione

1.1 Contesto e Motivazione

La narrazione interattiva e la generazione procedurale di contenuti rappresentano due ambiti di ricerca con profonde implicazioni sia nel campo dell'intrattenimento digitale che nello sviluppo di sistemi intelligenti. La creazione di storie coinvolgenti e strutturalmente coerenti richiede tradizionalmente un significativo investimento di tempo e competenze creative da parte di autori esperti. Al contempo, la pianificazione automatica offre strumenti formali potenti per garantire che le sequenze di eventi generate siano logicamente valide e raggiungano obiettivi predefiniti.

Negli ultimi anni, i Large Language Models (LLM) hanno dimostrato capacità sorprendenti nella generazione di contenuti creativi e nella comprensione del linguaggio naturale. Tuttavia, la loro applicazione alla creazione di narrazioni interattive strutturate presenta sfide significative: i modelli generativi tendono a produrre contenuti coerenti localmente ma possono mancare di consistenza globale, mentre i sistemi di pianificazione formale garantiscono correttezza logica a scapito della naturalezza narrativa.

QuestMasterAI nasce dall'intuizione che queste due approcci apparentemente contrastanti possano essere integrati in modo sinergico: i modelli generativi forniscono la creatività e la capacità di produrre narrazioni coinvolgenti, mentre la pianificazione automatica in PDDL (Planning Domain Definition Language) garantisce la coerenza strutturale e la validità logica delle quest generate.

1.2 Obiettivi del Progetto

Il progetto QuestMasterAI si propone di sviluppare un sistema completo per la generazione automatica di quest narrative interattive, combinando tecniche avanzate di intelligenza artificiale con metodologie formali di pianificazione. Gli obiettivi principali possono essere articolati come segue:

1. **Planning AI-Driven:** Realizzare una pipeline che trasformi input in linguaggio naturale in rappresentazioni formali PDDL, sfruttando le capacità generative dei moderni LLM per colmare il gap tra descrizioni narrative e specifiche formali;
2. **Validazione automatica:** Implementare un meccanismo robusto di validazione che utilizzi planner automatici (Fast-Downward) e validatori formali (VAL) per garantire che le

quest generate siano non solo narrative plausibili ma anche logicamente risolvibili;

3. **Correzione iterativa:** Sviluppare un Reflective Agent capace di analizzare errori di pianificazione e validazione, proponendo automaticamente correzioni ai file PDDL generati fino al raggiungimento di una soluzione valida;
4. **Arricchimento contestuale:** Integrare un sistema RAG (Retrieval-Augmented Generation) opzionale che permetta di arricchire la generazione con informazioni provenienti da documenti esterni, garantendo coerenza con lore preesistenti o specifiche di ambientazione;
5. **Human-in-the-Loop:** Fornire punti di intervento umano strategici nella pipeline, permettendo agli utenti di revisionare e modificare sia la lore generata che il piano di azioni prima della creazione della storia finale;
6. **Interfaccia accessibile:** Realizzare un'interfaccia web intuitiva basata su Flask che renda il sistema utilizzabile anche da utenti non tecnici, nascondendo la complessità dei formalismi sottostanti.

1.3 Contributi

Il presente lavoro fornisce i seguenti contributi principali:

- Una metodologia originale per la trasformazione automatica di descrizioni narrative in linguaggio naturale in rappresentazioni PDDL formalmente valide, mediata da una struttura intermedia di "lore" semi-strutturata;
- Un'architettura modulare che integra componenti eterogenei (LLM, planner PDDL, vector stores, interfaccia web) in una pipeline coerente e manutenibile;
- Un meccanismo di correzione automatica guidato da errori che analizza output di planner e validatori per proporre fix mirati ai file PDDL, riducendo significativamente l'intervento manuale necessario;
- Un'implementazione completa e funzionante del sistema, documentata in dettaglio e validata attraverso l'intera pipeline dalla generazione alla fruizione interattiva;
- Un'analisi approfondita delle tecnologie coinvolte e delle scelte architetturali, fornendo motivazioni tecniche per le decisioni di design adottate.

Capitolo 2

Analisi dei requisiti

2.1 Introduzione

Questo capitolo presenta l'analisi dei requisiti del sistema **QuestMasterAI**, un'applicazione software per la generazione automatica di quest narrative interattive. L'analisi copre requisiti funzionali e non funzionali, requisiti di sistema, i casi d'uso principali e i vincoli tecnici del progetto. L'obiettivo è fornire una descrizione completa delle funzionalità richieste, delle tecnologie coinvolte e delle modalità di interazione con il sistema, creando una base solida per la progettazione e l'implementazione.

2.2 Requisiti funzionali

I requisiti funzionali descrivono le funzionalità che il sistema **QuestMasterAI** deve fornire agli utenti. Ogni requisito è identificato con un codice del tipo **RFX** e include priorità, descrizione e sotto-requisiti tecnici. I requisiti sono organizzati seguendo il flusso logico del sistema:

- **RF1** – Generazione della lore narrativa
- **RF2** – Generazione dei file PDDL (domain e problem)
- **RF3** – Validazione PDDL con planner automatici
- **RF4** – Correzione automatica tramite Reflective Agent
- **RF5** – Validazione e approvazione del piano da parte dell'utente
- **RF6** – Generazione della storia narrativa finale
- **RF7** – Interfaccia utente Web (Flask)

2.2.1 RF1: Generazione della lore

Priorità: **Alta**.

Il sistema deve generare una lore narrativa strutturata in formato **JSON** a partire da un input testuale dell'utente. L'input utente è una descrizione libera della quest: il sistema deve supportare linguaggi naturali (italiano e inglese) e imporre vincoli di lunghezza (minimo 10 caratteri,

massimo 5000), oltre a controlli per mitigare la *prompt injection*. In presenza di pattern sospetti di *prompt injection* (ad esempio comandi che richiedono di ignorare istruzioni precedenti o di entrare in modalità privilegiata) il sistema deve sostituire l'output con una lore di default e registrare l'evento nei log per revisione.

Il sistema *RAG* è opzionale: quando abilitato carica documenti PDF indicati tramite variabile d'ambiente, estrae il testo con `PyPDFLoader`, suddivide il testo in chunk (configurazione tipica: chunk size 1000 caratteri con overlap 200) tramite `RecursiveCharacterTextSplitter`, calcola embeddings (ad esempio con Google Generative AI embeddings, modello `gemini-embedding-001`) e costruisce un *vector store* con **FAISS**. La ricerca recupera i tre chunk più rilevanti ($k=3$) e il contesto *RAG* viene limitato a una porzione utile del prompt (ad esempio massimo 2000 caratteri) per controllare latenza e costi.

L'output della generazione è un **JSON** con campi chiave quali `quest_description` (titolo, descrizione, obiettivo), `branching_factor` (min/max azioni per stato), `depth_constraints` (min/max passi), e `narrative_elements` (personaggi, luoghi, oggetti, azioni). Dopo la generazione la lore deve poter essere sottoposta a correzioni iterative in modalità *Human-in-the-Loop*: l'utente può proporre modifiche testuali che il sistema integra mantenendo la struttura del **JSON** e ripetendo le fasi successive solo a conferma dell'utente.

2.2.2 RF2: Generazione file PDDL

Priorità: **Alta**.

Dalla lore **JSON** devono essere prodotti i file `domain.pddl` e `problem.pddl`. Il *domain* deve dichiarare i requirements compatibili con il planner scelto (esempi: `:strips`, `:typing`, eventuali estensioni necessarie), i tipi (personaggi, luoghi, oggetti, ecc.), i predicati utili alla modellazione della lore e le azioni con parametri, precondizioni ed effetti. Il *problem* deve istanziare gli oggetti derivati dalla lore, definire lo stato iniziale e il goal coerente con l'obiettivo della quest. I file generati vengono salvati nella directory di output configurata (ad esempio `file_generati/`).

2.2.3 RF3: Validazione PDDL

Priorità: **Critica**.

Il processo di validazione consiste in due fasi concatenate: esecuzione del planner (**Fast-Downward**) per ottenere un piano operativo e validazione del piano con **VAL**. Lo script di validazione deve catturare output ed eventuali errori del planner, estrarre il piano di azioni e passarlo al validatore **VAL** che verifica la correttezza formale rispetto al *domain* e al *problem*. Se **Fast-Downward** non riesce a generare una soluzione o **VAL** segnala violazioni, il sistema deve attivare il *loop di correzione automatica*, fino a un massimo configurabile di tentativi — da noi impostato a **100** — in cui il Reflective Agent propone modifiche, i file **PDDL** vengono rigenerati e il processo di planning viene rieseguito.

2.2.4 RF4: Reflective Agent

Priorità: **Alta**.

Il *Reflective Agent* è un componente automatico che analizza messaggi di errore provenienti da **Fast-Downward** e **VAL**, identifica la tipologia di problema (predicati non definiti, mismatch di tipi, errori sintattici, incoerenze tra dominio e problema, o assenza di percorso valido) e invoca il modello **LLM** con un prompt specializzato contenente il contesto necessario (*lore*, *domain*, *problem* e messaggi d'errore) per ottenere proposte di correzione. Quando l'agent non è in grado

di determinare in modo univoco la correzione, il sistema richiede input all'utente (decisione se la correzione riguarda **DOMAIN** o **PROBLEM**) e ripete il ciclo fino a ottenere un file corretto.

2.2.5 RF5: Validazione del piano da parte dell'utente

Priorità: **Media**.

L'utente deve poter visualizzare il **piano** generato in formato leggibile (lista numerata di azioni con statistiche quali numero totale di azioni e lunghezza stimata) e accettarlo o richiedere modifiche. Se l'utente richiede correzioni, è prevista la ricezione di una descrizione testuale del problema che viene utilizzata per rigenerare *domain* e *problem* e per riavviare completamente la validazione con **Fast-Downward** e **VAL** fino all'approvazione.

2.2.6 RF6: Generazione della storia narrativa

Priorità: **Alta**.

Una volta validato e approvato il piano, il sistema deve leggere il piano, invocare il modello generativo per trasformare la sequenza di azioni in una *narrazione coerente* e arricchita, mantenendo la coerenza con la lore originale. La storia viene salvata in formato testuale e resa disponibile via endpoint web per download o visualizzazione.

2.2.7 RF7: Interfaccia utente Web (Flask)

Priorità: **Media**.

L'interfaccia principale è una web app **Flask** che espone endpoint **REST** per avviare le fasi salienti della pipeline: generazione lore, validazione **PDDL**, generazione storia ed endpoint di stato. Il frontend web fornisce moduli per l'inserimento dell'input quest, la visualizzazione della lore generata, la revisione del piano e la fruizione della storia finale. Il backend è avviato tramite lo script `run_flask.py`.

2.3 Requisiti non funzionali

I requisiti non funzionali definiscono le qualità e i vincoli del sistema in termini di:

- **Sicurezza;**
- **Affidabilità;**
- **Performance;**
- **Usabilità;**
- **Manutenibilità;**
- **Portabilità;**
- **Scalabilità.**

Ogni requisito è identificato con un codice del tipo **RNFX** e include una descrizione dettagliata delle aspettative di qualità.

2.3.1 Sicurezza (RNF1)

Il sistema deve mitigare attacchi di *prompt injection* mediante pattern matching e validazione degli input; applicare limiti di lunghezza sugli input; gestire in modo sicuro le **API key** tramite variabili d'ambiente e file di configurazione non versionati; sanitizzare e validare l'output dei modelli prima di effettuare parsing strutturato.

2.3.2 Affidabilità (RNF2)

Le operazioni critiche devono essere protette da gestione delle eccezioni e meccanismi di retry. Il sistema prevede un fallback a una lore di default in caso di errori di generazione e mantiene un limite massimo di tentativi per la validazione PDDL. Il logging dettagliato degli errori supporta la diagnosi e la riproducibilità dei casi falliti.

2.3.3 Performance (RNF3)

Per contenere latenza e costi sono previsti timeout su chiamate LLM configurabili, limitazione del contesto RAG (es. 2000 caratteri), caching degli embeddings ove possibile e chunking ottimizzato dei documenti di riferimento (1000 caratteri, overlap 200).

2.3.4 Usabilità (RNF4)

La web app deve fornire un'interazione chiara, con messaggi di stato facilmente interpretabili e feedback progressivo per operazioni lunghe. L'interfaccia è in grado di supportare input in italiano; il layout e lo styling sono pensati per facilitare la revisione della lore, del piano e della storia finale.

2.3.5 Manutenibilità (RNF5)

L'architettura del codice è modulare: la logica di generazione contenuti risiede in `file_generation/`, la validazione e la correzione in `correction_and_validation/`, e l'interfaccia web in `flask_app/`. Le dipendenze sono gestite tramite `requirements.txt` e la configurazione è esternalizzata in file `.env`. Docstring e commenti devono accompagnare le funzioni critiche per facilitare il refactoring.

2.3.6 Portabilità (RNF6)

Il codice è pensato per essere eseguito su Linux, macOS e Windows; le dipendenze sono installabili con pip e si raccomanda l'utilizzo di ambienti virtuali. I path devono essere trattati in modo cross-platform e gli eseguibili esterni (Fast-Downward, VAL) devono essere installati separatamente.

2.3.7 Scalabilità (RNF7)

La struttura modulare consente di sostituire o aggiornare componenti (diversi LLM, altri planner PDDL, vector store alternativi). La componente RAG è opzionale e disabilitabile se il deployment richiede minori risorse.

2.4 Casi d'uso

I casi d'uso descrivono gli scenari tipici di interazione con il sistema. Sono presentati tre casi principali di complessità crescente.

2.4.1 UC1: Generazione quest completa

Attore primario: utente.

Precondizioni: sistema configurato e API key valide.

Flusso: l'utente inserisce la descrizione della quest; il sistema genera la lore JSON; l'utente può approvare o richiedere modifiche; dopo approvazione il sistema genera domain e problem in PDDL; il planner esegue Fast-Downward; il piano risultante viene validato con VAL; il piano viene mostrato all'utente che può approvarlo e, dopo approvazione, il sistema genera la storia narrativa finale che viene mostrata all'utente.

In caso di errore durante la pianificazione o la validazione, il flusso alternativo prevede l'attivazione del reflective agent per correggere automaticamente i file PDDL oppure la richiesta di intervento da parte dell'utente, seguita da una nuova iterazione del ciclo di validazione.

2.4.2 UC2: Correzione manuale PDDL

Attore primario: utente esperto.

Precondizioni: esistenza di un piano generato ma giudicato non soddisfacente.

Flusso: il sistema mostra il piano; l'utente fornisce una descrizione testuale delle modifiche; il sistema rigenera domain e problem e riesegue la validazione; il nuovo piano viene mostrato e il ciclo continua fino all'approvazione.

2.4.3 UC3: Utilizzo RAG per contesto esterno

Attore primario: utente.

Precondizioni: file PDF di riferimento configurato tramite la variabile d'ambiente RAG_FILE.

Flusso: l'utente invia una query; il sistema carica il PDF, crea gli embeddings, recupera i tre chunk più rilevanti tramite FAISS e include il contesto risultante nel prompt LLM; la lore generata incorpora le informazioni estratte dal documento.

2.5 Vincoli e dipendenze

Il progetto dipende da servizi esterni e componenti installati localmente: API di **Google Generative AI** (necessarie per gli embeddings e alcune funzionalità), il planner **Fast-Downward** e il validatore **VAL** che devono essere installati sul sistema di esecuzione, e **FAISS** come *vector store* per RAG. Sono imposti vincoli operativi quali un limite al numero di tentativi di validazione (ad es. **100**) e vincoli sulla lunghezza degli input (10–5000 caratteri) e sul contesto RAG (es. massimo 2000 caratteri). I file generati sono salvati localmente nella cartella di output configurata (ad es. `file_generati/`).

Capitolo 3

Tecnologie utilizzate

3.1 Panoramica

QuestMasterAI integra componenti eterogenei per trasformare un input in linguaggio naturale in una rappresentazione formale del problema di pianificazione e, successivamente, in una narrazione interattiva coerente con il piano prodotto. In questo capitolo descriviamo le principali tecnologie adottate e le motivazioni delle scelte, privilegiando modularità, affidabilità e riproducibilità.

3.2 Core language e orchestrazione

3.2.1 Python

Python è il linguaggio principale del progetto e svolge il ruolo di collante tra i moduli del sistema. Viene utilizzato per orchestrare il workflow, interfacciarsi con servizi esterni (LLM ed embeddings), serializzare i dati e integrare strumenti di pianificazione eseguiti come processi separati. L'adozione di librerie standard come `subprocess` e `pathlib` contribuisce a mantenere il codice portabile e robusto, riducendo dipendenze superflue e semplificando la manutenzione.

3.3 Pipeline LLM e orchestrazione AI

3.3.1 LangChain

Per strutturare la pipeline basata su Large Language Models, il sistema utilizza **LangChain**. La scelta è motivata dalla disponibilità di astrazioni che semplificano l'invocazione dei modelli, la composizione di catene di elaborazione (prompting, post-processing e controlli) e l'integrazione con componenti di retrieval documentale. In questo modo l'architettura resta modulare e consente di sostituire con relativa facilità il provider del modello o la strategia di recupero del contesto senza impatti estesi sul resto del codice.

3.3.2 LLM: Google Gemini 2.5 Flash

Il modello linguistico impiegato è **Gemini 2.5 Flash**, integrato tramite il modulo `langchain-google-genai`. Nel progetto viene configurato con una *temperatura* pari a **0.6**, valore scelto per bilanciare due esigenze in parte in conflitto: da un lato una creatività sufficiente a generare una

lore e una narrazione coinvolgenti, dall'altro una coerenza logica adeguata a produrre PDDL formalmente corretto e stabile rispetto a vincoli e struttura del dominio.

3.4 RAG e gestione della conoscenza

Per arricchire le generazioni con contesto specifico (ad esempio lore preesistente, vincoli di ambientazione o regole), è stato implementato un approccio di *Retrieval-Augmented Generation* (RAG). In questa configurazione, porzioni di documenti esterni vengono recuperate in base alla similarità semantica con la richiesta e reiniettate nel prompt, riducendo allucinazioni e migliorando la coerenza rispetto alle fonti.

3.4.1 Caricamento e segmentazione documenti (PyPDF)

I documenti della knowledge base in formato PDF vengono processati tramite PyPDFLoader. Il testo estratto viene poi suddiviso con RecursiveCharacterTextSplitter utilizzando chunk da **1000** caratteri e un overlap di **200** caratteri. Questa scelta riduce la perdita di informazione nei punti di taglio e preserva il contesto locale, rendendo più efficace il recupero di passaggi pertinenti.

3.4.2 Vector store e retrieval (FAISS + embeddings)

La ricerca semantica è realizzata con **FAISS** nella variante `faiss-cpu`. I chunk vengono indicizzati attraverso embeddings prodotti dal modello `gemini-embedding-001` di Google, in modo da supportare query basate sul significato e non sulla sola corrispondenza lessicale. Il recupero avviene in modo dinamico al momento della richiesta dell'utente e consente di costruire un contesto mirato da inserire nel prompt, ottimizzando l'uso della context window e mantenendo l'informazione rilevante.

3.5 Pianificazione formale e validazione

3.5.1 Standard PDDL

La pianificazione automatica viene modellata tramite **PDDL** (*Planning Domain Definition Language*), che consente di rappresentare in modo dichiarativo il dominio (predicati e azioni con precondizioni ed effetti) e l'istanza del problema (oggetti, stato iniziale e obiettivi). In QuestMasterAI, i file PDDL sono generati dinamicamente a partire dagli elementi narrativi prodotti o estratti dalla pipeline LLM, così da trasformare la quest in un problema risolvibile da un planner domain-independent.

3.5.2 Planner: Fast-Downward

Per risolvere i problemi di pianificazione viene utilizzato **Fast-Downward**, integrato tramite subprocess. L'esecuzione è isolata in un processo esterno e protetta da un timeout di sicurezza pari a **300** secondi; stdout e stderr vengono catturati per consentire analisi e debugging. È stato selezionato l'algoritmo A* con euristica *blind*, scelta orientata alla *completezza*: se il problema ammette una soluzione, la procedura di ricerca la troverà, sebbene con un costo computazionale potenzialmente superiore rispetto a euristiche più informative.

3.5.3 Validazione: VAL

Trovare un piano non è sufficiente: è necessario verificarne la correttezza rispetto alla semantica PDDL. A questo scopo il sistema integra **VAL** (*Plan Validator*), che simula l'esecuzione della sequenza di azioni verificando la soddisfacibilità delle precondizioni a ogni step e controllando che lo stato finale soddisfi gli obiettivi. L'output di VAL viene analizzato per individuare errori ricorrenti, come precondizioni non soddisfatte o goal mancati, e per produrre un feedback strutturato utile a guidare eventuali correzioni automatiche dei file PDDL.

3.6 Infrastruttura web e configurazione

3.6.1 Backend web: Flask

L'interfaccia utente è servita da **Flask**, scelto per la sua leggerezza e per la facilità con cui consente di esporre endpoint web e API. Nel progetto, le route rappresentano le fasi principali del workflow, permettendo al frontend di attivare operazioni di generazione, validazione e pianificazione e di ricevere risposte strutturate senza imporre vincoli architetturali troppo rigidi.

3.6.2 Persistenza e gestione configurazione

I dati scambiati tra i moduli vengono serializzati in **JSON**, scelta che favorisce ispezionabilità, debugging e interoperabilità tra componenti. Le configurazioni sensibili o dipendenti dall'ambiente, come chiavi API, percorsi degli eseguibili Fast-Downward/VAL e secret key di Flask, vengono gestite tramite variabili d'ambiente con supporto a file `.env`, in modo da separare configurazione e codice e ridurre il rischio di esposizione accidentale di credenziali.

Capitolo 4

Sviluppo del progetto

Questo capitolo descrive in modo operativo l'implementazione del progetto, collegando i requisiti funzionali alla struttura del codice e fornendo le indicazioni necessarie per comprendere come i componenti cooperano lungo la pipeline di generazione. L'applicazione è stata progettata attorno a un backend web basato su **Flask**.

4.1 Componente RAG e generazione della lore

La logica di **Retrieval-Augmented Generation (RAG)** e la generazione della lore sono raccolte nel modulo `file_generation/lore_generation.py`.

In questa fase il sistema:

- i Acquisisce opzionalmente documentazione di supporto in formato **PDF**;
- ii Costruisce un archivio vettoriale per il recupero semantico;
- iii Compone un prompt che combina richiesta utente e contesto recuperato, producendo una lore strutturata utilizzabile dagli stadi successivi.

Quando è attiva la modalità **RAG**, il caricamento del **PDF** è eseguito tramite `PyPDFLoader`, che restituisce una lista di oggetti `Document` contenenti testo e metadati. Il testo viene spezzato in segmenti (*chunk*) e trasformato in *embedding*; tali *embedding* vengono indicizzati in un *vector store* (**FAISS**) e interrogati tramite un *retriever* configurato in modalità *similarity*.

Nella fase di creazione del *vector store*, **FAISS** organizza i *chunk* di testo precedentemente estratti e processati insieme ai loro *embedding* generati dal modello di Google. Questo *vector store* viene poi configurato come *retriever* con una strategia di ricerca basata sulla similarità, parametrizzato per recuperare i tre *chunk* più rilevanti rispetto alla query dell'utente. Durante il recupero del contesto, il sistema invoca il *retriever* con l'input dell'utente, estrae il contenuto testuale dai documenti recuperati e li combina in un unico contesto contestuale limitato a 2000 caratteri per rispettare i limiti del modello e mantenere la rilevanza dell'informazione.

4.1.1 Strategie di Prompting e Sicurezza

Il sistema fa largo uso di tecniche avanzate di *prompt engineering* per massimizzare la qualità degli output. Il *few-shot learning* fornisce al modello esempi concreti dell'output desiderato,

guidandolo verso il formato corretto. Il *defensive prompting* implementa protezioni contro tentativi di *prompt injection* che potrebbero compromettere il funzionamento del sistema. La richiesta di *structured output* assicura che il modello generi sempre **JSON** valido e parsabile. Infine, tecniche di *chain-of-thought* guidano esplicitamente il ragionamento del modello attraverso problemi complessi, migliorando la qualità delle soluzioni proposte.

Per prevenire abusi e attacchi di *prompt injection*, il sistema implementa controlli di sicurezza che identificano pattern malevoli nell'input utente, come istruzioni che tentano di far ignorare al modello le sue istruzioni precedenti, richieste di dimenticare tutto, tentativi di cambiare il ruolo del modello, o riferimenti a tecniche di *jailbreak* e *modalità sviluppatore*. Quando questi pattern vengono rilevati, l'input viene rifiutato prima ancora di raggiungere il modello.

```
1 self.embeddings = GoogleGenerativeAIEmbeddings(model="gemini-embedding-001")
2 loader = PyPDFLoader(self.pdf_path)
3 documents = loader.load()
4 splits = text_splitter.split_documents(documents)
5 self.vector_store = FAISS.from_documents(splits, self.embeddings)
6 self.retriever = self.vector_store.as_retriever(
7     search_type="similarity",
8     search_kwargs={"k": 3}
9 )
```

Listing 4.1: Esempio di costruzione del vector store e retriever (estratto).

Nel codice è presente una gestione che intercetta l'assenza di **FAISS** e, in quel caso, disabilita la pipeline **RAG** facendo proseguire l'esecuzione senza *retrieval*. In tal modo il sistema rimane eseguibile anche in ambienti dove *faiss-cpu* non è installabile, mantenendo invariata l'interfaccia di generazione della lore e differenziando soltanto la qualità/ricchezza del contesto fornito al modello.

4.1.2 Struttura della lore come ponte verso il PDDL

Durante lo sviluppo è emerso che una lore puramente narrativa rende instabile la conversione in **PDDL**, perché introduce ambiguità su entità, stati e azioni. Per questo motivo l'output della fase di *lore generation* è stato impostato come descrizione controllata del mondo: vengono esplicitati gli oggetti rilevanti, le relazioni iniziali e gli obiettivi, e le azioni vengono descritte con vincoli che possano essere tradotti in precondizioni ed effetti. In questo modo la lore funziona da interfaccia interna tra linguaggio naturale e rappresentazione formale, riducendo il numero di correzioni necessarie nelle fasi successive.

4.2 Generazione PDDL e validazione

La trasformazione della lore in **PDDL** è implementata nei moduli `domain_generation.py` e `problem_generation.py`. Il primo produce il `domain.pddl`, definendo tipi, predicati e azioni con precondizioni ed effetti; il secondo produce il `problem.pddl`, istanziando oggetti, stato iniziale e obiettivi.

Un aspetto rilevante dello sviluppo è la gestione dell'output del modello in forma “*machine-oriented*”: i prompt di generazione vengono progettati in modo da ottenere sezioni ben delimitate e facilmente parsabili, così che l'applicazione possa estrarre e scrivere i file **PDDL** evitando

trasformazioni manuali. L'obiettivo implementativo è ridurre la variabilità dell'output e rendere riproducibile il passaggio `lore` → PDDL.

Dopo la generazione, il progetto esegue una fase di controllo che mira a intercettare due classi di problemi: errori sintattici (file PDDL non analizzabile) ed errori semantici o di pianificazione (dominio coerente ma non risolvibile, o soluzione prodotta non valida). In caso di fallimento, il sistema costruisce un feedback descrittivo dell'errore e attiva un ciclo di correzione automatica che rigenera o modifica i file PDDL fino al raggiungimento di un piano valido o al superamento di un numero massimo di tentativi. Questa logica implementa di fatto il requisito di correzione iterativa senza richiedere intervento manuale immediato.

4.2.1 Esecuzione del planner e validazione del piano

Una volta scritti `domain.pddl` e `problem.pddl`, la pipeline invoca un planner esterno (**Fast Downward**) per trovare una sequenza di azioni che soddisfi gli obiettivi. Il piano calcolato viene successivamente validato tramite **VAL**, che controlla che le azioni siano applicabili nello stato corrente e che gli effetti conducano effettivamente allo stato obiettivo. La scelta di separare “trovare un piano” e “validare un piano” riduce i falsi positivi dovuti a domini incompleti o a piani non applicabili in pratica.

L'integrazione di **Fast-Downward** nel workflow verifica innanzitutto l'esistenza dei file **PDDL** necessari, evitando errori di esecuzione dovuti a path non validi. Il sistema costruisce poi il comando appropriato includendo i path ai file `domain` e `problem` e la configurazione dell'algoritmo di ricerca. L'esecuzione avviene tramite `subprocess` con cattura dell'output standard ed errori, impostando un *timeout* di sicurezza di 300 secondi. Dopo l'esecuzione, il sistema analizza l'output per determinare se una soluzione è stata trovata, facendo il parsing di specifici messaggi nell'output prodotto. In caso di successo, il risultato include un flag di successo e il path al file del piano, mentre in caso di fallimento vengono inclusi i messaggi di errore dettagliati per facilitare il debugging.

VAL rappresenta il secondo livello critico di validazione nel workflow di **QuestMasterAI**. Mentre **Fast-Downward** verifica che esista almeno una soluzione al problema di pianificazione, **VAL** garantisce che la soluzione trovata sia formalmente corretta secondo le regole della semantica **PDDL**. Solo dopo che entrambe le validazioni hanno successo, il piano viene accettato e può procedere alla fase di generazione della storia. Questa validazione a due livelli assicura la massima qualità e correttezza formale dei piani generati.

4.2.2 Esempio minimale di dominio

Di seguito è riportato un estratto semplificato di dominio che illustra la corrispondenza tra concetti narrativi e PDDL. L'obiettivo non è mostrare il dominio completo, ma chiarire come una singola azione venga resa in termini di precondizioni ed effetti.

```
1 (:action move
2   :parameters (?c - character ?from - location ?to - location)
3   :precondition (and (at ?c ?from) (connected ?from ?to))
4   :effect (and (not (at ?c ?from)) (at ?c ?to)))
```

Listing 4.2: Estratto PDDL (esempio).

Questo tipo di rappresentazione permette al planner di esplorare sequenze di azioni applicabili a partire dallo stato iniziale e raggiungere gli obiettivi definiti nel `problem` file.

4.2.3 Classi di errori e strategia di correzione

Il ciclo di correzione implementato nel sistema integra strettamente **Fast-Downward** e **VAL** in un processo iterativo. Il sistema esegue prima **Fast-Downward** per ottenere un piano. Se **Fast-Downward** fallisce, attiva immediatamente il workflow di correzione passando gli errori al **Reflective Agent** che tenta di correggere il **PDDL**. Se **Fast-Downward** ha successo ma **VAL** rileva che il piano non è valido, il sistema estrae gli errori specifici di validazione e li fornisce al **Reflective Agent** per una correzione mirata. Questo processo continua iterativamente fino a quando entrambe le validazioni hanno successo o viene raggiunto il numero massimo di tentativi, proteggendo da cicli infiniti di correzione.

Nel corso dello sviluppo si sono osservate ricorrenze negli errori prodotti dai modelli generativi. Una prima classe riguarda inconsistenze dichiarative, ad esempio oggetti utilizzati ma non dichiarati oppure predicati presenti nelle azioni ma non definiti nel dominio. Una seconda classe riguarda incompatibilità di tipo, quando un parametro viene usato come se appartenesse a un tipo diverso da quello dichiarato. Una terza classe riguarda la pianificabilità, in cui dominio e problema sono sintatticamente validi ma non consentono di raggiungere l'obiettivo per mancanza di azioni o per vincoli troppo restrittivi.

La correzione automatica è stata implementata come ciclo guidato da errori: l'output degli strumenti (planner e validator) viene ridotto a un feedback compatto e riutilizzabile e quindi utilizzato per rigenerare il **PDDL**. Il ciclo termina quando viene trovato un piano validato oppure quando viene raggiunto un numero massimo di iterazioni, condizione necessaria per evitare esecuzioni indefinite.

4.3 Esecuzione tramite Flask

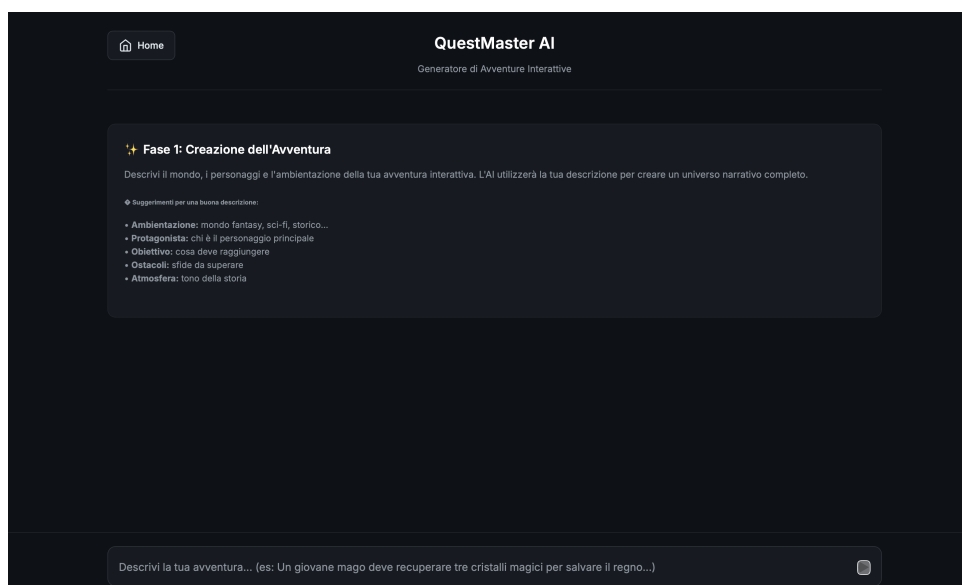


Figura 4.1: La home page di QuestMasterAI

Il sistema **QuestMasterAI** utilizza un pattern architetturale basato su stati per gestire il flusso dell'applicazione. Ogni stato rappresenta una fase specifica del workflow, dalla creazione iniziale fino al gameplay interattivo. Il routing dinamico permette di navigare tra questi stati in modo fluido e controllato attraverso URL parametrici che identificano lo stato corrente.

L'applicazione espone **API REST** con endpoint **JSON** per operazioni asincrone che non richiedono il reload completo della pagina. Questi endpoint gestiscono operazioni computazionalmente intensive come la generazione della lore, la validazione **PDDL** e la creazione della storia, restituendo risultati strutturati in formato **JSON** che il frontend **JavaScript** può processare dinamicamente.

Le sessioni **Flask** mantengono lo stato dell'applicazione per ciascun utente, memorizzando informazioni critiche come lo stato corrente del workflow, la lore generata, il piano di azioni corrente e i progressi dell'utente nel gameplay. Una chiave segreta configurabile garantisce la sicurezza crittografica delle sessioni, proteggendo da manipolazioni non autorizzate dei dati di sessione.

L'esecuzione operativa del progetto avviene tramite un'applicazione **Flask** che espone un'interfaccia web per inserire l'input dell'utente e avviare la pipeline. **Flask** svolge il ruolo di *orchestratore*: riceve la richiesta, avvia la generazione della lore, chiama la generazione del **PDDL**, attiva la pianificazione e la validazione e, a conclusione, restituisce il risultato all'interfaccia.

Una parte significativa dello sviluppo ha riguardato la gestione di esecuzioni potenzialmente lunghe (chiamate *LLM*, planner esterno e loop di correzione). Per questo motivo è importante che l'API definisca chiaramente quali file vengono prodotti, dove vengono salvati e come viene tracciato l'avanzamento dell'elaborazione, in modo da rendere il comportamento ispezionabile e riproducibile anche in fase di debug.

4.3.1 Interazione Frontend-Backend

L'applicazione web espone un'interazione che consente di inviare una richiesta testuale e ottenere come risultato gli artefatti generati dalla pipeline. Dal punto di vista dello sviluppo, è stato importante mantenere un contratto stabile tra frontend e backend, in modo che le iterazioni sulla logica interna non richiedessero modifiche frequenti all'interfaccia. Un esempio semplificato di payload, utile per chiarire il tipo di input gestito dal backend, è mostrato di seguito.

```
1 {  
2   "prompt": "Crea una quest ambientata in un tempio sommerso...",  
3   "use_rag": true  
4 }
```

Listing 4.3: Esempio di payload (semplificato).

4.4 Flusso operativo tipico

L'architettura di **QuestMasterAI** integra tutte le tecnologie descritte in una pipeline coerente e ben orchestrata. Il flusso inizia con l'input dell'utente, che viene raccolto attraverso l'interfaccia basata su **Flask**. Questa prima interazione stabilisce il tema e il contesto della quest che l'utente desidera creare.

Il sistema **RAG** opzionale entra in azione caricando documenti **PDF** di riferimento attraverso **PyPDF**, che estrae il testo preservando la struttura del documento. Questi contenuti vengono trasformati in *embedding* vettoriali utilizzando il modello di Google integrato in **LangChain**, e **FAISS** organizza questi vettori in un indice efficiente per la ricerca di similarità. Quando l'utente fornisce input, il sistema recupera automaticamente i *chunk* più rilevanti che verranno utilizzati per arricchire il contesto fornito al modello linguistico.

La generazione della lore costituisce la prima fase creativa del processo. **LangChain** orchestra l'interazione con **Gemini 2.5 Flash**, costruendo prompt sofisticati che includono tecniche di *defensive prompting* per prevenire abusi e *few-shot learning* per guidare il formato dell'output. Il modello genera una struttura **JSON** completa che descrive personaggi, luoghi, oggetti e trama, che viene poi salvata utilizzando le funzionalità **JSON** native di **Python**.

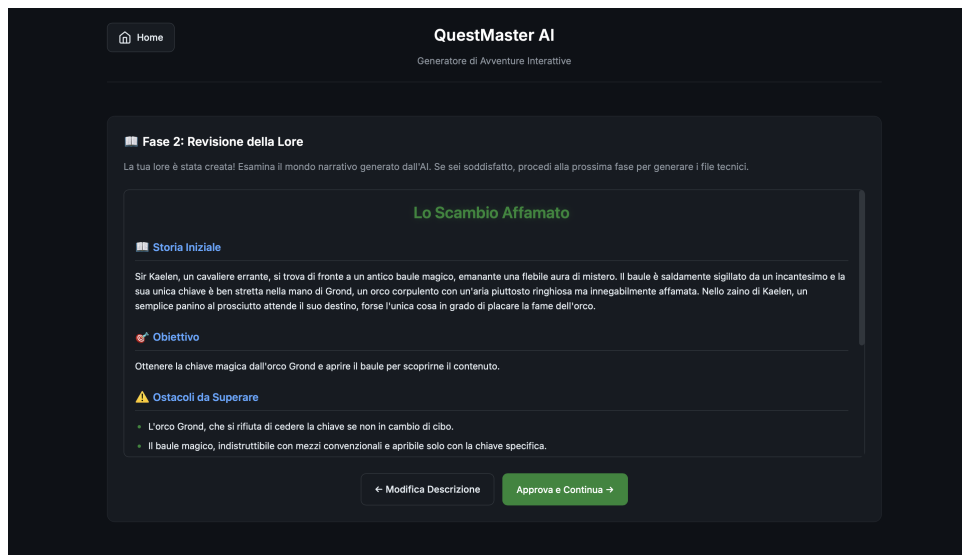


Figura 4.2: Schermata di revisione della lore

La trasformazione della lore in **PDDL** rappresenta il passaggio cruciale dalla narrazione alla pianificazione formale. **LangChain** e **Gemini** collaborano per tradurre gli elementi narrativi in costrutti **PDDL** formali, generando sia il domain file che definisce le regole del mondo sia il problem file che specifica l'istanza concreta da risolvere. La gestione dei file attraverso **pathlib** assicura che questi artefatti vengano salvati nelle posizioni corrette e siano accessibili alle fasi successive.

Il processo di validazione implementa un controllo di qualità rigoroso a due livelli. **Fast-Downward** viene eseguito tramite subprocess per tentare di trovare una soluzione al problema di pianificazione. L'output del planner viene attentamente analizzato per estrarre informazioni su successo o fallimento e identificare errori specifici. Se **Fast-Downward** produce un piano, **VAL** entra in azione per verificarne la correttezza formale, simulando l'esecuzione step-by-step e assicurando che tutte le precondizioni siano rispettate e il goal sia raggiunto.

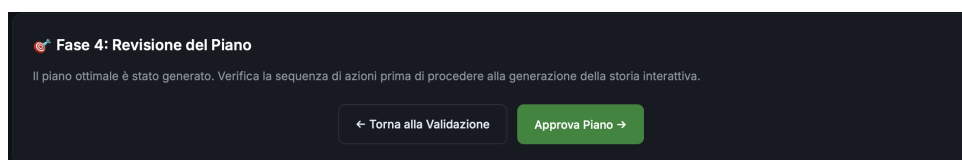


Figura 4.3: Schermata di approvazione per il piano generato

Quando vengono rilevati errori, il **Reflective Agent** implementato attraverso **LangChain** e **Gemini** analizza i messaggi di errore dettagliati forniti dai tool di validazione. Il modello identifica la natura specifica del problema, propone correzioni mirate al codice **PDDL**, e rigenera i file corretti. Nei casi in cui il sistema non può risolvere automaticamente l'ambiguità, viene implementato un meccanismo di *Human-in-the-Loop* che richiede chiarimenti all'utente prima di procedere.

Una volta ottenuto un piano valido, la fase di generazione della storia converte la sequenza astratta di azioni in una narrativa coinvolgente. **LangChain** e **Gemini** trasformano il piano lineare in un grafo di scelte interattive, dove ogni nodo rappresenta una situazione narrativa e le scelte disponibili al giocatore. Questa struttura a grafo viene serializzata in **JSON** e resa disponibile per il gameplay.

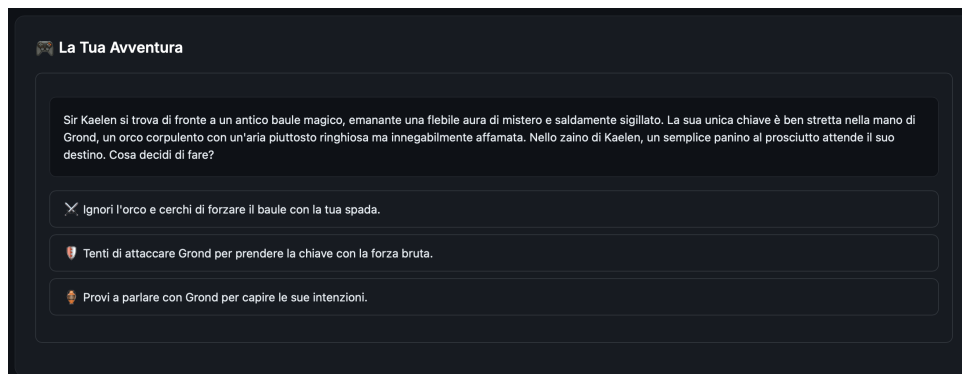


Figura 4.4: Schermata del gioco

Il gameplay finale utilizza **Flask** e **JavaScript** per creare un'esperienza interattiva fluida. Il rendering dinamico dei nodi permette di presentare il testo narrativo e le scelte disponibili. **JavaScript** gestisce le interazioni dell'utente, inviando le scelte selezionate al backend che naviga nel grafo della storia, recupera il nodo successivo e aggiorna l'interfaccia. Questo ciclo continua fino a quando il giocatore raggiunge uno dei nodi finali della narrazione.

Gemini 2.5 Flash è impiegato in quattro fasi critiche del workflow. La generazione della lore richiede al modello di creare strutture **JSON** narrative complete e coerenti che descrivono personaggi, luoghi, oggetti e trama. Nella fase di generazione **PDDL**, il modello trasforma questa lore in codice **PDDL** formale, traducendo elementi narrativi in predicati, azioni e obiettivi pianificabili. Il **Reflective Agent**, implementato sempre tramite **Gemini**, analizza gli errori di validazione forniti dai tool di planning e propone correzioni mirate al codice **PDDL**. Infine, nella generazione della storia, il modello converte il piano astratto di azioni in una narrativa coinvolgente e ramificata con scelte significative per il giocatore.

4.4.1 Interazioni dei componenti

Le dipendenze tra i diversi componenti del sistema formano una rete complessa ma ben strutturata. La generazione della lore dipende fondamentalmente da **Python** come linguaggio base, **LangChain** per l'orchestrazione e **Gemini** per l'intelligenza generativa, costituendo il core AI del sistema. Il sistema **RAG** richiede **PyPDF** per l'estrazione di testo dai documenti, **FAISS** per l'indicizzazione e la ricerca vettoriale, e gli *embedding* di Google per la rappresentazione semantica dei contenuti.

La generazione di **PDDL** si appoggia a **Python** per l'esecuzione, **LangChain** per la gestione dei prompt complessi, e **Gemini** per la traduzione creativa ma formalmente corretta dalla narrativa al codice **PDDL**. La validazione dipende criticamente da **Fast-Downward** e **VAL** come tool esterni, eseguiti e monitorati attraverso il modulo `subprocess` di **Python**.

Il **Reflective Agent** combina **LangChain** per l'orchestrazione del ciclo di correzione, **Gemini** per l'analisi intelligente degli errori e la proposta di fix, e **JSON** per la persistenza delle versioni

successive del **PDDL** corretto. L'interfaccia web richiede **Flask** come framework backend, **Jinja2** per il templating HTML, e **JavaScript** per l'interattività client-side.

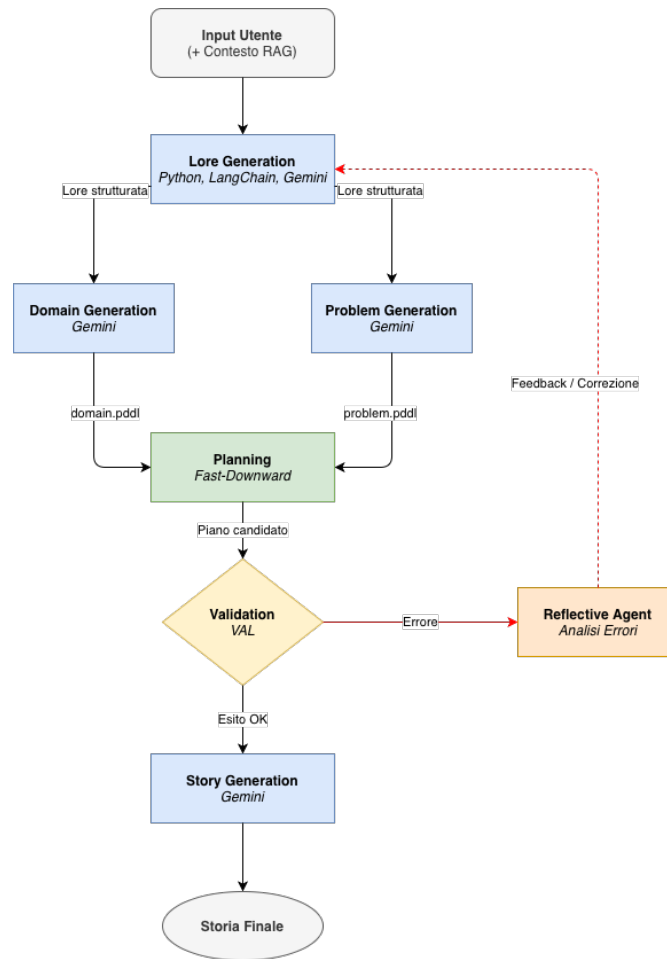


Figura 4.5: Grafico che rappresenta le varie interazioni tra i componenti

4.4.2 Artefatti prodotti lungo la pipeline

L'esecuzione produce una serie di artefatti intermedi che rendono il flusso ispezionabile. In particolare, la lore generata viene salvata per permettere una verifica indipendente dalla generazione **PDDL**; i file `domain.pddl` e `problem.pddl` costituiscono l'input per la pianificazione; il piano risultante viene memorizzato per poter essere validato e, successivamente, usato come vincolo narrativo nella generazione finale. La presenza di tali artefatti si è rivelata fondamentale in fase di debugging, perché consente di identificare se un fallimento nasce dalla generazione, dalla conversione o dalla pianificazione.

4.5 Mapping requisiti funzionali — codice

Il collegamento tra requisiti funzionali e implementazione è diretto lungo la pipeline. La generazione della lore (RF1) è incapsulata nel modulo di *lore generation* e nelle funzioni che compongono prompt e, se attivo, *retrieval*. La generazione dei file **PDDL** (RF2) corrisponde ai moduli di domain e problem generation. La pianificazione e la validazione (RF3) sono realizzate tramite l'invocazione degli strumenti esterni e l'interpretazione dei loro output. La correzione automatica (RF4) è implementata come ciclo di rigenerazione guidato dagli errori. La com-

ponente *Human-in-the-Loop* (RF5), dove presente nell'interfaccia, si inserisce tra validazione e generazione finale, consentendo di fermare o accettare un piano. La storia finale (RF6) è prodotta dall'ultimo stadio di generazione, che utilizza lore e piano validato. L'interfaccia web (RF7) è implementata nell'applicazione **Flask** che orchestra e presenta l'intero flusso.

Req.	Implementazione (moduli principali)
RF1	file_generation/lore_generation.py (generazione lore, prompt, RAG opzionale)
RF2	file_generation/domain_generation.py, file_generation/problem_generation.py
RF3	Invocazione Fast Downward e validazione con VAL (wrappers/script di esecuzione)
RF4	Ciclo di rigenerazione guidato da errori (feedback da planner/validator)
RF6	Modulo di story generation basato su lore e piano validato
RF7	Applicazione Flask come orchestratore e interfaccia web

Tabella 4.1: Tracciamento requisiti funzionali → moduli.

Capitolo 5

Conclusioni

5.1 Risultati Ottenuti

Il progetto QuestMasterAI ha dimostrato la fattibilità di un'integrazione efficace tra modelli linguistici generativi e sistemi di pianificazione formale per la creazione automatica di narrazioni interattive. L'implementazione realizzata consente di trasformare descrizioni testuali in linguaggio naturale in quest completamente giocabili, passando attraverso una rappresentazione formale in PDDL che garantisce coerenza logica e risolubilità.

La pipeline sviluppata ha raggiunto gli obiettivi prefissati, integrando con successo componenti eterogenei in un flusso di lavoro coerente. Il sistema RAG opzionale si è rivelato efficace nell'arricchire la generazione con informazioni contestuali provenienti da documenti esterni, migliorando la qualità e la specificità delle quest generate. L'architettura modulare adottata ha facilitato lo sviluppo iterativo e permette di sostituire o aggiornare singoli componenti senza impattare l'intera pipeline.

Il meccanismo di Reflective Agent per la correzione automatica degli errori PDDL rappresenta uno dei contributi più significativi del lavoro. L'analisi automatica degli output di Fast-Downward e VAL, combinata con la capacità dei modelli generativi di proporre correzioni mirate, ha ridotto drasticamente la necessità di intervento manuale, rendendo il sistema utilizzabile anche da utenti non esperti in pianificazione formale. I cicli iterativi di generazione-validazione-correzione convergono tipicamente entro poche iterazioni, dimostrando l'efficacia dell'approccio.

L'interfaccia web basata su Flask fornisce un punto di accesso intuitivo all'intero sistema, implementando efficacemente il paradigma Human-in-the-Loop che permette agli utenti di intervenire nei momenti critici del processo di generazione. La separazione tra backend di elaborazione e frontend di presentazione ha inoltre facilitato il testing e il debugging delle componenti individuali.

5.2 Sfide e Limitazioni

Nonostante i risultati positivi, l'implementazione ha evidenziato alcune limitazioni significative che meritano discussione.

5.2.1 Qualità e Coerenza della Generazione

La qualità delle quest generate dipende criticamente dalla capacità del modello linguistico di produrre PDDL sintatticamente corretto e semanticamente coerente. Sebbene il meccanismo di correzione automatica riesca a risolvere molti errori comuni (predicati non definiti, inconsistenze di tipo, precondizioni mancanti), esistono casi in cui il sistema entra in cicli di correzione che non convergono entro il limite massimo di tentativi configurato. Questi casi richiedono tipicamente una revisione manuale più approfondita della lore o dei vincoli imposti.

La traduzione dalla lore JSON al PDDL rappresenta un passaggio critico dove possono introdursi ambiguità o interpretazioni errate. Una lore puramente narrativa tende a produrre PDDL instabile, motivo per cui il sistema è stato progettato per generare lore semi-strutturate che fungono da ponte tra linguaggio naturale e formalismo. Tuttavia, questo approccio può limitare la creatività narrativa e richiede un bilanciamento attento.

5.2.2 Performance e Scalabilità

Le chiamate ripetute ai modelli LLM durante la fase di correzione iterativa possono comportare latenze significative, specialmente quando il numero di correzioni necessarie è elevato. Inoltre, i costi associati all'utilizzo delle API di modelli commerciali come Google Gemini possono diventare rilevanti in scenari di utilizzo intensivo. Il sistema attualmente non implementa meccanismi di caching sofisticati che potrebbero ridurre queste limitazioni.

La dimensione e la complessità delle quest generabili è limitata dalle capacità dei planner PDDL utilizzati. Problemi di pianificazione particolarmente complessi possono richiedere tempi di esecuzione proibitivi o fallire completamente a trovare soluzioni entro i timeout configurati.

5.2.3 Dipendenze Esterne

Il sistema dipende criticamente da servizi e tool esterni: le API di Google Generative AI per gli embeddings e la generazione, Fast-Downward e VAL come eseguibili locali, FAISS per il vector store. Questa architettura distribuita introduce punti di fallimento multipli e complica il deployment in ambienti con requisiti di sicurezza stringenti o senza accesso a internet.

La necessità di installare e configurare correttamente Fast-Downward e VAL rappresenta una barriera all'adozione, specialmente su sistemi Windows dove la compilazione di questi tool può risultare problematica. Una containerizzazione completa del sistema tramite Docker potrebbe alleviare questi problemi.

5.3 Lavori Futuri

Il lavoro svolto apre diverse direzioni di ricerca e sviluppo future che potrebbero significativamente estendere le capacità del sistema.

5.3.1 Miglioramenti alla Generazione

L'adozione di tecniche di prompt engineering più sofisticate, come il *chain-of-thought reasoning* esplicito o l'utilizzo di esempi *few-shot* dinamicamente selezionati dal retriever RAG, potrebbe migliorare la qualità della generazione PDDL riducendo il numero di iterazioni necessarie. L'integrazione di modelli specializzati per la generazione di codice formale potrebbe inoltre ridurre gli errori sintattici.

L'implementazione di un sistema di feedback esplicito dall'utente sulle quest generate permetterebbe di costruire un dataset di esempi di alta qualità che potrebbe essere utilizzato per il fine-tuning di modelli specifici per questo dominio applicativo. Questo approccio di apprendimento continuo potrebbe migliorare progressivamente le performance del sistema.

5.3.2 Valutazione e Validazione

Uno studio sistematico della qualità delle quest generate, condotto attraverso valutazioni con utenti reali e metriche quantitative (complessità narrativa, bilanciamento, varietà delle scelte), fornirebbe insight preziosi sull'efficacia del sistema e identificherebbe aree specifiche di miglioramento.

L'implementazione di un framework di testing automatico che generi e validi un gran numero di quest diverse permetterebbe di caratterizzare statisticamente le performance del sistema, identificando categorie di input che tendono a fallire o richiedere molte iterazioni.

5.3.3 Architettura e Deployment

La migrazione verso un'architettura a microservizi con componenti containerizzati (generazione lore, validazione PDDL, story generation) migliorerebbe la scalabilità e permetterebbe di distribuire il carico computazionale su più istanze. L'introduzione di una coda di messaggi per gestire richieste asincrone renderebbe il sistema più robusto e capace di gestire picchi di traffico.

L'implementazione di meccanismi di caching intelligenti per lore, PDDL e piani già generati potrebbe ridurre significativamente latenze e costi, specialmente in scenari dove utenti richiedono variazioni minori di quest esistenti.

5.4 Considerazioni Finali

QuestMasterAI dimostra che l'integrazione tra Intelligenza Artificiale Generativa e pianificazione formale rappresenta un approccio promettente per la creazione automatica di contenuti interattivi di qualità. Il sistema sviluppato costituisce un proof-of-concept funzionante che evidenzia sia le potenzialità che le sfide di questo paradigma.

La combinazione di creatività algoritmica e rigore formale apre prospettive interessanti non solo nel dominio dei videogiochi e dell'intrattenimento interattivo, ma anche in applicazioni educative, di training professionale e di storytelling adattivo. Le tecniche sviluppate potrebbero essere adattate a domini diversi dove è necessario generare sequenze di eventi logicamente coerenti arricchite con elementi narrativi.

Il lavoro svolto ha permesso di acquisire competenze approfondite nell'integrazione di tecnologie eterogenee, nella gestione di pipeline complesse di elaborazione e nell'applicazione pratica di tecniche avanzate di intelligenza artificiale e pianificazione automatica. Le scelte architetture adottate, documentate in dettaglio in questa relazione, potranno servire da riferimento per progetti futuri in ambiti correlati.

In conclusione, QuestMasterAI rappresenta un passo significativo verso la democratizzazione della creazione di contenuti interattivi complessi, rendendo accessibili capacità creative che tradizionalmente richiedevano competenze tecniche specializzate. Il sistema realizzato, seppur perfettibile, costituisce una base solida per ulteriori sviluppi e sperimentazioni in questo affascinante dominio di ricerca.